

PFC AI Design Agent DesignState Backend — Pre-Implementation Review

This document covers every file that will change, every file that will not change, the rollback switch, and all risk areas. Read this before any code is written.

GUI changes?	ZERO — App.tsx, client.ts, and all components are untouched.
API changes?	ZERO — No endpoint signatures change. No request/response shapes change.
Rollback?	1-line feature flag in feature_flags.py — already exists in codebase.
New files?	3 new files only. No existing file is structurally changed.
Risk level?	LOW — DesignState is validation-only in Phase 1. Existing dict flow unchanged.

Part 1 — How the Current State Flow Works (baseline before any change)

Understanding the existing flow is the foundation of this review. Every API call today follows the same pattern: the frontend sends a **state: dict** and the backend reads fields from it with **state.get(...)**. Nothing validates the shape of that dict — it simply passes through.

Step	Frontend (React/TypeScript)	Backend (Python/FastAPI)
App.tsx stores state	graphState: Record<string,unknown> null Approved designs stored as separate fields: approvedInductorDesign approvedCapacitorDesign	No backend storage — stateless. Each request carries full state.
API call (every step)	client.ts calls e.g.: step7RunSizing({ state: graphState, material_key, ...}) step15CapacitorDesign({ state: graphState })	Endpoint receives state: dict Extracts fields: state.get('intake', {}) state.get('topology_specific_inputs', {}) No shape validation — any dict accepted
Response handling	Frontend merges response into AppState via setS() GraphState updated with r.state from server Step results stored in approvedXxxDesign	Endpoint returns JSON dict. No schema enforced on output shape.
Report generation	Passes graphState + approvedDesign in one payload step15GenerateReport({ state, approved_design, step15_result })	PDF generator reads from passed dict. No DesignState schema involved.

KEY INSIGHT

The frontend never validates the state shape. The backend never validates the state shape. Both sides trust that the dict contains the right fields. DesignState is the schema that makes this contract explicit — but it does NOT change the wire format at all.

Part 2 — File-by-File Impact Table

Every file in the codebase is categorised below. The two columns that matter most are **Change Type** and **GUI Impact**.

A. New Files (3 total — all additive, nothing existing is changed)

File	Location	Purpose	GUI Impact	Rollback
schemas.py	backend/app/schemas.py	Pydantic DesignState model. Defines the shape of the state dict as a versioned Python class. Used internally by backend agents to validate what they read/write. Extra fields always allowed (model_config extra='allow').	ZERO	Delete file
DESIGN_STATE.md	docs/DESIGN_STATE.md	Human documentation listing every DesignState field, which agent owns it, and its type. Documentation only.	ZERO	Delete file
DesignState.ts	frontend/src/types/DesignState.ts	TypeScript interface mirroring schemas.py. PURE TYPE FILE — zero runtime code. The frontend already uses Record<string,unknown> for graphState. This is documentation in type form only.	ZERO	Delete file

B. Modified Files (2 files — small additions only)

File	What Changes	Lines Added	GUI Impact	Rollback
feature_flags.py backend/app/config/	Add one new flag: enable_design_state_validation: bool = False Default is FALSE — no behaviour changes until explicitly set to True.	+1 line	ZERO	Remove the 1 added line
main.py backend/app/	ONE optional validation helper: _validate_state(state: dict) → None Called at the top of each endpoint ONLY if enable_design_state_validation is True. If flag is False (default): function is never called. Existing endpoint code is not touched.	+8 lines (optional helper only)	ZERO	Remove the 8 helper lines or set flag=False

C. Untouched Files (GUI guaranteed — these files do not change at all)

File	Reason Not Changed	GUI Impact
frontend/src/App.tsx	AppState interface and all handlers remain exactly as-is. DesignState is a backend concept — App.tsx never knows it exists.	ZERO
frontend/src/api/client.ts	No endpoint signatures change. No request/response shapes change. All POST calls continue sending { state: dict, ...params }.	ZERO
frontend/src/components/Step7Wizard.tsx	Component logic, props, enrichResult(), wireKey(), fmtMat() all untouched.	ZERO
frontend/src/components/Step15Capacitor.tsx Step15Wizard.tsx	selected_cap pattern, all verify flows untouched.	ZERO
frontend/src/components/ControlDesign.tsx and all other components	No component is touched. DesignState is invisible to the frontend.	ZERO
backend/app/mode_b/step7_magnetic_calc.py	DesignResult dataclass unchanged. Calculation logic unchanged.	ZERO
backend/app/mode_b/generate_steps13_14.py generate_combined_report.py generate_report.py generate_step15.py	PDF generators unchanged. They continue reading from the passed dicts.	ZERO
backend/app/mode_b/step15_capacitor.py step16_control_design.py step8_time_domain.py	All calculation engines unchanged.	ZERO
backend/app/magnetics/db.py magnetics/extractor.py magnetics/schema.py	MagneticsDB unchanged.	ZERO

All JS studio HTML files review_magnetics.html control_design.html bode_plot.html	Standalone HTML tools — completely isolated from backend schema changes.	ZERO
--	--	-------------

Part 3 — Rollback Mechanism

The rollback has two layers. Layer 1 is a feature flag — instant, no restart needed. Layer 2 is file deletion — removes DesignState entirely as if it was never added.

Layer 1 — Feature Flag (instant rollback, 30 seconds)

The existing file `backend/app/config/feature_flags.py` already has this pattern. We add exactly one line:

backend/app/config/feature_flags.py — after change (1 line added, highlighted)

```
class FeatureFlags(BaseModel):
    enable_guardrail_v2: bool = True
    enable_supply_chain_agent: bool = True
    enable_magnetic_design_v2: bool = True
    # ... existing flags ...

    # DesignState validation – OFF by default
    # Set True to enable schema validation on incoming state dicts
    # Set False to revert to current behaviour instantly (no restart needed)
    enable_design_state_validation: bool = False # <-- THE ENTIRE ROLLBACK SWITCH
```

Layer 2 — File Deletion (full rollback, removes DesignState entirely)

File to Delete	Effect
backend/app/schemas.py	DesignState Pydantic model gone. No imports broken (only imported behind feature flag).
docs/DESIGN_STATE.md	Documentation gone. No code impact.
frontend/src/types/DesignState.ts	TypeScript type gone. No runtime code — nothing breaks.
Remove 1 line from feature_flags.py	enable_design_state_validation field removed.
Remove 8 lines from main.py	_validate_state() helper removed. Endpoints return to their current state exactly.

ROLLBACK GUARANTEE

After full rollback, the codebase is byte-for-byte identical to today EXCEPT for those 3 new files being absent and 9 lines being removed from 2 files. The GUI, all API endpoints, all report generators, and all calculation engines are in exactly the same state as today.

Part 4 — Risk Analysis (what could go wrong and how it is mitigated)

#	Risk	Likelihood	Impact	Mitigation
R1	Pydantic validation rejects a frontend payload that has extra or renamed fields	Low	Medium (endpoint 500s)	Mitigation 1: model_config = ConfigDict(extra='allow') on DesignState. This means extra fields are accepted silently — only missing required fields raise errors. Mitigation 2: Validation is behind feature flag (default OFF). Mitigation 3: If it fires, flip flag to False in <30 seconds.
R2	A new agent or endpoint in future adds DesignState but uses the wrong field name	Medium	Low (wrong value, not a crash)	DESIGN_STATE.md documents every field with its owner. This is the same risk that exists today without DesignState — it is made explicit and checked

R3	Import error — schemas.py imports typing that is not installed in the venv	Very Low	High (backend crash on startup)	schemas.py uses only Pydantic (already installed) and Python stdlib. No new packages are added. Syntax check runs before any commit.
R4	DesignState.ts TypeScript type conflicts with existing Record<string,unknown> usage	Very Low	Low (type warning, not runtime error)	DesignState.ts is a separate type file. It is never imported by App.tsx or client.ts unless the developer explicitly adds that import. In Phase 1, it is documentation only.
R5	schema.py name conflict with existing app/engines/state_space/schemas.py	Very Low	Low (import error)	Place file at backend/app/design_state.py instead of schemas.py to avoid any naming clash. This is noted as an action item in Part 5.

Part 5 — Phased Implementation Plan (when to do what, in what order)

Each phase is independently deployable. Phase 1 carries zero risk because the feature flag defaults to OFF. You can complete Phase 1, run the app for a week to confirm nothing changed, then proceed to Phase 2.

Phase 1 — Schema Only (zero risk, zero GUI impact)

#	Action	Detail
1	Create backend/app/design_state.py	Pydantic DesignState model. All fields Optional with defaults. model_config = ConfigDict(extra='allow'). No imports from this file yet.
2	Create docs/DESIGN_STATE.md	Field ownership table. Every field from the current state dict documented.
3	Create frontend/src/types/DesignState.ts	TypeScript interface. Not imported anywhere yet — documentation only.
4	Add to feature_flags.py	enable_design_state_validation: bool = False — default OFF, no behaviour change.
5	Add _validate_state() to main.py	8-line helper: if FEATURE_FLAGS.enable_design_state_validation: validate(state). The helper is defined but the flag is OFF so it is never called.
6	Syntax check + run app	python -c 'import ast; ast.parse(...)' on all modified files. Start uvicorn, hit /health, confirm nothing changed.

Phase 2 — Opt-in Validation (enable flag, test endpoints one by one)

#	Action	Detail
1	Set enable_design_state_validation = True in test.py	Restart the app with the flag ON. Hit each endpoint from the frontend. Confirm no 422 or 500 errors. If any appear, fix the field mapping.
2	Fix any field name mismatches	These are documentation bugs, not code bugs. Update DESIGN_STATE.md fields to match what the frontend actually sends.
3	Set flag default to True when all endpoints pass	Only after every endpoint has been manually tested.

Phase 3 — Documentation Agent reads DesignState (new feature, existing code untouched)

#	Action	Detail
1	Create backend/app/mode_b/documentation_agent.py	New file only. Reads DesignState fields and generates report sections. No existing file is modified.
2	Add POST /mode-b/generate-chapter endpoint	New endpoint only. The 'Generate Full Report' button in the GUI calls this. Existing report endpoints (generate-full-report, generate-report) are untouched.
3	Add a button in DonePanel or Step7 UI	Small additive change — one button added. No existing button or flow changes.

Part 6 — One Pre-Implementation Correction Found During Review

During the file audit, one naming conflict was found that must be resolved BEFORE implementation begins.

Issue	Original Plan	Corrected Plan	Reason
File name conflict	Create backend/app/schemas.py	Create backend/app/design_state.py	backend/app/engines/state_space/schemas.py already exists. Also backend/app/llm/schemas.py exists. Two schemas.py files in the same project causes ambiguous imports. design_state.py is unambiguous and self-describing.
Reference in DOCUMENT_STATE.md	References 'schemas.py'	References 'design_state.py'	Consistent with the corrected file name.
Reference in feature_flags.py	No import needed	No import needed	Feature flag file already exists — we just add one bool field to it.

Part 7 — Decision Summary (answer before implementation begins)

Decision	Recommendation	Why
Proceed with Phase 1?	YES — safe to implement now	Phase 1 is 3 new files + 1 flag (OFF by default) + 8 helper lines. Zero behaviour change. Full rollback in <2 minutes.
Proceed with Phase 2?	YES — but only after Phase 1 is confirmed working for 1+ run	Phase 2 turns validation ON. Test each endpoint manually first. The flag lets you revert instantly if anything misbehaves.
Proceed with Phase 3?	YES — after Phase 2 is stable and you explicitly want the Documentation Agent	Phase 3 is a new feature. It adds a new file and a new endpoint. Existing code paths are not touched — the Documentation Agent is additive, not a replacement.
Change App.tsx?	NO — never	DesignState is a backend concept. The GUI state machine in App.tsx works correctly today and must not be touched for this feature.
Change any existing endpoint?	NO — never for Phase 1 or 2	Existing endpoints get only an optional validation call (behind the feature flag). Their logic, signatures, and response shapes are unchanged.

FINAL VERDICT

This change is SAFE to implement if the phased plan is followed. The worst-case scenario at any phase is: one feature flag flip + app restart = back to today's codebase. No GUI changes. No API contract changes. No report changes. The DesignState schema is purely a backend internal contract that agents use to validate what they read from the state dict.